



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

IoT Challenge 2 - Exercise

TESI DI LAUREA MAGISTRALE IN
TELECOMMUNICATION ENGINEERING

Author: **Hong CHEN** [REDACTED]
Mengzhen LI [REDACTED]

Student ID: [REDACTED] & [REDACTED]

Advisor: Prof. Name Surname

Co-advisors:

Academic Year: 2025-04

Contents

Contents	i
1 Part 2 Exercise	1
1.1 Scenario Setup and Assumptions	1
1.1.1 CoAP	2
1.1.2 MQTT	2
1.2 EQ1: Energy Calculation	3
1.2.1 EQ1(a): CoAP Protocol Energy Calculation	3
1.2.2 EQ1(b): MQTT Protocol Energy Calculation	4
1.3 EQ2: Two Energy Optimization Schemes	6
1.3.1 Energy Optimization Strategy 1: Data Aggregation and Transmission Every 30 Minutes	7
1.3.2 Optimization Strategy 2: Offloading Average Temperature Calculation from Valve to Raspberry Pi	8

1 | Part 2 Exercise

1.1. Scenario Setup and Assumptions

Sensor: Every 5 minutes, the temperature sensor measures the temperature and sends the data, battery-powered, Wi-Fi enabled. **Valve:** The valve receives the temperature data from the sensor every 30 minutes, computes the average, and decides whether to open or close, battery-powered, Wi-Fi enabled. **Raspberry Pi:** Raspberry Pi, connected to the power grid, supports only MQTT for communication.

COAP		MQTT	
GET Request	60B	Subscribe	58B
GET Response	55B	Sub Ack	52B
PUT Request	77B	Publish	68B
PUT Response	58B	Pub Ack	51B
Empty ACK	14B	Connect	54B
		Publish	68B
		Connect Ack	47B
		Ping Req	52B
		Ping Resp	48B

Table 1.1: Message Sizes (in bytes)

ETX	ERX
50 nJ/bit	58 nJ/bit

Table 1.2: Energy Consumption Parameters (in nJ/bit)

Valve Processing Energy: Every 30 minutes, the valve consumes energy $E_c = 2.4$ mJ for computing the average temperature.

1.1.1. CoAP

CoAP (Constrained Application Protocol) is a lightweight application-layer protocol designed specifically for constrained devices and low-power networks. It is based on UDP and follows a request/response model, making it ideal for peer-to-peer communication in IoT systems.

In the most energy-efficient mode, the process works as follows: The sensor sends a

Feature	Confirmable Mode	Non-Confirmable Mode
Message Ack	Requires receiver to acknowledge the message (via ACK or response).	No acknowledgment needed, receiver may choose not to respond or send an empty ACK.
Reliability	Higher reliability, as sender waits for acknowledgment to confirm successful reception.	Lower reliability, suitable for situations where message loss can be tolerated.
Energy Consumption	Higher energy consumption	Lower energy consumption

Table 1.3: CoAP Mode

PUT Request message to the valve, such as to update the temperature. Upon receiving the PUT Request, the valve only sends an empty ACK message for acknowledgment, without sending a full PUT Response. The sensor receives the ACK and considers the request successfully received. This approach avoids transmitting large data or response messages from the receiver, which significantly reduces the power consumption of the communication.

1.1.2. MQTT

The client establishes a TCP connection with the Broker. A client (e.g., Raspberry Pi) publishes a message to a topic. Other clients subscribed to the topic (e.g., valve) receive the message. QoS levels determine whether acknowledgments are required.

QoS (Quality of Service) is a mechanism in the MQTT protocol that defines the reliability level of message delivery. Different QoS levels involve different handshaking processes between the sender and the receiver, balancing data reliability and energy consumption.

QoS 0 - (At most once): This is the most energy-efficient mode. The sender sends the message once without retries, and the receiver does not acknowledge. Message loss is possible.

QoS 1 - (At least once): Ensures the message is received at least once. The receiver sends a PUBACK as an acknowledgment. If no PUBACK is received, the sender will retry,

which may cause duplication.

In this project, QoS 0 is chosen due to its suitability for energy-sensitive, battery-powered devices like sensors and valves. QoS 0 requires only a single PUBLISH message without waiting for acknowledgment, which significantly reduces power consumption and is ideal for low-power IoT applications. The advantages of this approach include the simplest communication process with no handshaking, resulting in the lowest energy consumption. However, the trade-off is that messages may be lost, and the receiver cannot confirm delivery, making this mode unsuitable for applications requiring reliable data transmission.

1.2. EQ1: Energy Calculation

The sensor sends a PUT request every 5 minutes and receives a PUT response.

In 24 hours, the sensor will send 24 hours / 5 minutes = 288 PUT requests.

The processing cost on the valve to compute the average temperature every 30 minutes is **Ec = 2.4mJ**. The valve performs 48 calculations per day (once every 30 minutes).

$$E_{Calculate} = 48E_c = 48 \times 2.4mJ = 115.2mJ$$

1.2.1. EQ1(a): CoAP Protocol Energy Calculation

Device	Action(Tx/Rx)	Size (B)	Counts	$E_{total}(mJ)$
Sensor	TX PUT Req	77	288	8.87
Valve	RX PUT Req	77	288	10.29
Valve	TX Empty ACK	14	288	1.61
Sensor	RX Empty ACK	14	288	1.87

Table 1.4: CoAP

The energy consumption for sending 77 bytes of data is calculated as:

$$\begin{aligned}
 E_{Sensor-per} &= E_{Sensor-TXPUTReq} + E_{Sensor-RXEmptyACK} \\
 &= 77bytes \times 8bits/byte \times 50nJ/bit + 14 \times 8bits/ \times 58nJ/bit \\
 &= 3.08 \times 10^{-2}mJ + 0.65 \times 10^{-2}mJ \\
 &= 3.73 \times 10^{-2}mJ \text{ (per transmission)}
 \end{aligned} \tag{1.1}$$

Energy Consumption of the Valve:

$$\begin{aligned}
E_{Valve-per} &= E_{Valve-RXPUTReq} + E_{Valve-TXEmptyACK} \\
&= 77bytes \times 8 \times 58nJ/bit + 14bytes \times 8 \times 50nJ/bit \\
&= 3.57 \times 10^{-2}mJ + 0.56 \times 10^{-2}mJ \\
&= 4.13 \times 10^{-2}mJ(per\ transmission)
\end{aligned} \tag{1.2}$$

The total energy consumption calculation is as follows:

$$\begin{aligned}
E_{total_COAP} &= 288 \times (E_{Sensor-per} + E_{Valve-per}) + E_{Calculate} \\
&= 288 \times (3.73 \times 10^{-2}mJ + 4.13 \times 10^{-2}mJ) + 115.2mJ \\
&= 22.63mJ + 115.2mJ \\
&= \mathbf{137.84mJ}
\end{aligned} \tag{1.3}$$

In CoAP Non-Confirmable mode, the sensor only sends PUT requests, and the valve only receives data without requiring ACK confirmation, achieving the lowest energy consumption of **137.84 mJ**.

1.2.2. EQ1(b): MQTT Protocol Energy Calculation

Calculate the energy for establishing the connection between the Valve and Raspberry Pi, once.

$$\begin{aligned}
E_{Conn-Valve} &= E_{Valve-TXCon} + E_{Valve-RXConnACK} \\
&= 54 \times 8 \times E_{TX} + 47 \times 8 \times E_{RX} \\
&= 2.16 \times 10^{-2}mJ + 2.18 \times 10^{-2}mJ \\
&= \mathbf{4.34 \times 10^{-2}mJ}
\end{aligned} \tag{1.4}$$

Calculate the energy for Valve subscribing to the Raspberry Pi temperature message, once.

$$\begin{aligned}
E_{Subscribe} &= E_{Valve-TXSub} + E_{Valve-RXSubACK} \\
&= 58 \times 8 \times E_{TX} + 52 \times 8 \times E_{RX} \\
&= 2.32 \times 10^{-2}mJ + 2.41 \times 10^{-2}mJ \\
&= \mathbf{4.73 \times 10^{-2}mJ}
\end{aligned} \tag{1.5}$$

Calculate the energy for Raspberry Pi publishing to Valve, every 5 minutes after connection.

$$\begin{aligned}
E_{Valve \leftrightarrow RasPi-Comm.} &= 288 \times E_{Valve-RXPub} \\
&= 288 \times 68 \times 8 \times E_{RX} \\
&= \mathbf{9.09mJ}
\end{aligned} \tag{1.6}$$

Calculate the total energy consumption of communication between Valve and Raspberry Pi.

$$\begin{aligned}
 E_{Valve-RasPi-Total} &= E_{Connect} + E_{Subscribe} + E_{Valve \leftrightarrow RasPi-Comm.} + E_{Compute} \\
 &= 4.34 \times 10^{-2}mJ + 4.73 \times 10^{-2}mJ + 9.09mJ + 115.2mJ \\
 &= \mathbf{124.3807mJ}
 \end{aligned} \tag{1.7}$$

Calculate the energy for establishing the connection between Sensor and Raspberry Pi, once.

$$\begin{aligned}
 E_{Connect-Sensor} &= E_{Sensor-TXConn} + E_{Sensor-RXConACK} \\
 &= 54 \times 8 \times E_{TX} + 47 \times 8 \times E_{RX} \\
 &= 2.16 \times 10^{-2}mJ + 2.18 \times 10^{-2}mJ \\
 &= \mathbf{4.34 \times 10^{-2}mJ}
 \end{aligned} \tag{1.8}$$

Calculate the energy for Sensor publishing to Raspberry Pi, every 5 minutes after connection.

$$\begin{aligned}
 E_{Sensor \leftrightarrow RasPi-Comm.} &= 288 \times E_{RasPi-RXPub} \\
 &= 288 \times 68 \times 8 \times E_{RX} \\
 &= \mathbf{7.8336mJ}
 \end{aligned} \tag{1.9}$$

Calculate the total energy consumption of communication between Sensor and Raspberry Pi.

$$\begin{aligned}
 E_{Sensor-RasPi-total} &= E_{Connect-Sensor} + E_{Sensor \leftrightarrow RasPi-Comm.} \\
 &= 4.34 \times 10^{-2}mJ + 7.8336mJ \\
 &= \mathbf{7.877mJ}
 \end{aligned} \tag{1.10}$$

Calculate the total energy consumption of the system in MQTT communication mode.

$$\begin{aligned}
 E_{MQTT-Total} &= E_{Valve-RasPi-Total} + E_{Sensor-RasPi-Total} \\
 &= 124.3807mJ + 7.877mJ \\
 &= \mathbf{132.2577mJ}
 \end{aligned} \tag{1.11}$$

As shown in Table 1.5 and Table 1.6, the Valve and Sensor communicate with the Raspberry Pi for data transmission. The total energy consumption of both parts for a 24-hour period is **132.2577 mJ**.

Concluding Analysis: According to the calculation and analysis, under the premise of selecting the mode that minimizes energy consumption, the energy consumption of CoAP mode is slightly higher than that of MQTT mode. The energy consumption in both modes is primarily caused by the energy consumption of the Valve module, which calculates the

Device	Action(Tx/Rx)	Message Type	Size (B)	Counts	$E_{\text{total}}(mJ)$
Connection stage					
Sensor	Tx	CONNECT	54	1	2.16×10^{-2}
Sensor	Rx	CONNACK	47	1	2.18×10^{-2}
Communication stage					
Sensor	Tx	PUBLISH	68	288	7.83
Total (Sensor): 7.877 mJ					

Table 1.5: Sensor's Energy Consumption

Device	Action(Tx/Rx)	Message Type	Size (B)	Counts	$E_{\text{total}}(mJ)$
Connection stage					
Valve	Tx	CONNECT	54	1	2.16×10^{-2}
Valve	Rx	CONNACK	47	1	2.18×10^{-2}
Subscribe stage					
Valve	Tx	Subscribe	58	1	2.32×10^{-2}
Valve	Rx	Sub Ack	52	1	2.41×10^{-2}
Communication stage					
Valve	Rx	Publish	68	288	9.09
Valve	Compute Avg Temp	NaN	-	48	115.2
Total (Valve): 124.3807 mJ					

Table 1.6: Valve's Energy Consumption in MQTT Mode

average temperature every 30 minutes.

1.3. EQ2: Two Energy Optimization Schemes

Publish Packet:

Publish Packet	Detial
Fixed Header	Control Package Type = 3 (PUBLISH)
	QoS=0 $\xrightarrow{\text{OPT}}$ QoS=1
	Remaining Length
Variable Header	Topic
	Identifier
Payload	Payload = Temp(8Bytes) $\xrightarrow{\text{OPT}}$ 48Bytes

Table 1.7: MQTT Mode PUBLISH Message Packet Structure and Optimization

1.3.1. Energy Optimization Strategy 1: Data Aggregation and Transmission Every 30 Minutes

In this scenario, the Raspberry Pi acts as the MQTT broker between the temperature sensor and the valve. To optimize energy consumption, the sensor aggregates temperature data over 30 minutes before sending it. Instead of sending data every 5 minutes, the sensor stores it locally and sends 6 data points together in one Publish message every 30 minutes. The Raspberry Pi then forwards this aggregated data to the valve in a single Publish message, reducing overhead from multiple topic headers.

This method reduces the number of Publish messages being sent, which directly reduces energy consumption.

Device	Action(Tx/Rx)	Message Type	Size (B)	Counts	$E_{\text{total}}(mJ)$
Connection stage					
Sensor	TX	CONNECT	54	1	2.16×10^{-2}
Sensor	RX	CONNACK	47	1	2.18×10^{-2}
Communication stage					
Sensor	TX(Every 30 min)	PUBLISH	$60+8 \times 6$	48	2.0736
Sensor	RX(Every 30 min)	Pub ACK	51	48	1.1359
Optimized -Total (Sensor): 3.2529 mJ					

Table 1.8: EQ2: Optimized Sensor's Energy Consumption in MQTT Mode

Device	Action(Tx/Rx)	Message Type	Size (B)	Counts	$E_{\text{total}}(mJ)$
Connection stage					
Valve	Tx	CONNECT	54	1	2.16×10^{-2}
Valve	Rx	CONNACK	47	1	2.18×10^{-2}
Subscribe stage					
Valve	Tx	Subscribe	58	1	2.32×10^{-2}
Valve	Rx	Sub Ack	52	1	2.41×10^{-2}
Communication stage					
Valve	RX(Every 30 min)	Publish	$60+8 \times 6$	48	2.4054
Valve	TX(Every 30 min)	Pub ACK	51	48	1.1359
Valve	Compute Avg Temp	NaN	-	48	115.2
Optimized - Total (Valve): 118.832 mJ					

Table 1.9: EQ2: Optimized Valve's Energy Consumption in MQTT Mode

Each Publish message has a fixed overhead of 60 bytes for the MQTT header and topic. By aggregating the data, we save this overhead multiple times. For instance, instead of sending 6 separate Publish messages, the sensor sends only 1, thus saving $60 \text{ bytes} \times 5 = 300 \text{ bytes}$ of overhead.

$$\begin{aligned}
E_{\text{Opt-MQTT-Tot}} &= E_{\text{Opt-Valve-Tot}} + E_{\text{Opt-Sensor-Tot}} \\
&= 118.832mJ + 3.2529mJ \\
&= \mathbf{122.0849mJ}
\end{aligned} \tag{1.12}$$

To ensure that each message is reliably transmitted and successfully received, the MQTT protocol typically uses the QoS (Quality of Service) mechanism.

Concluding Analysis:

In this optimization scheme, to ensure that the aggregated temperature message sent every 30 minutes is correctly received, we adopt QoS level 1 in the MQTT protocol, which requires that every PUBLISH message must be followed by a PUBACK acknowledgment. After optimization of the packet transmission, the fixed sending fields were saved. The total energy consumption over 24 hours for the first method, where data is aggregated and sent every 30 minutes, is **122.0849 mJ**, mainly saving energy from the non-payload fields of the PUBLISH message. This optimization results in a total energy saving of **132.2577 mJ - 122.0849 mJ = 10.1728 mJ** every 24 hours.

1.3.2. Optimization Strategy 2: Offloading Average Temperature Calculation from Valve to Raspberry Pi

In the original system design, the valve receives 6 temperature readings every 30 minutes and performs the average temperature calculation locally. Since the valve is battery-powered, this computation introduces additional energy consumption.

By offloading this computation to the Raspberry Pi, the valve only needs to receive a single pre-computed average value every 30 minutes, reducing both message reception and computation energy costs.

Energy Consumption and Saving Estimation:

$$\begin{aligned}
E_{\text{Opt2-Valve-Comm-Tot}} &= 48 \times (68 \times 8 \times E_{RX} + 51 \times 8 \times E_{TX}) \\
&= 48 \times (3.16 \times 10^{-2}mJ + 2.04 \times 10^{-2}mJ) \\
&= \mathbf{2.496mJ}
\end{aligned} \tag{1.13}$$

$$\begin{aligned}
E_{\text{Opt2-Valve-Tot}} &= E_{\text{Conn-Valve}} + E_{\text{Subscribe}} + E_{\text{Opt2-Valve-Comm-Tot}} \\
&= 4.34 \times 10^{-2}mJ + 4.73 \times 10^{-2}mJ + 2.496mJ \\
&= \mathbf{2.5867mJ}
\end{aligned} \tag{1.14}$$

$$\begin{aligned}
E_{\text{Opt2-Tot}} &= E_{\text{Opt-Sensor-Tot}} + E_{\text{Opt2-Valve-Tot}} \\
&= 3.2529\text{mJ} + 2.5867\text{mJ} \\
&= \mathbf{5.8396\text{mJ}}
\end{aligned} \tag{1.15}$$

The total energy saved by the second optimization over 24 hours is:

$$\begin{aligned}
E_{\text{Opt2-Saved}} &= E_{\text{Opt-MQTT-Tot}} - E_{\text{Opt2-Tot}} \\
&= 122.0849\text{ mJ} - 5.8396\text{ mJ} \\
&= \mathbf{116.2453\text{ mJ}}
\end{aligned} \tag{1.16}$$

Concluding analysis:

After optimization, the total daily energy consumption of the sensor and valve is approximately **5.8396 mJ**, significantly lower than before. As a result, this strategy saves around **116.2453 mJ** of energy, which is highly valuable for battery-powered devices.

